Research Intake 2026

Day 29

Introduction to CNNs

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

The future of AI is bright, and you can be part of it!

1 From Handcrafted Features to Learned Features

This day marks the module's central turning point. Days 26 through 28 developed the classical computer vision pipeline in full — image representation, preprocessing, and a complete detect-describe-match-filter-estimate feature pipeline built entirely from hand-designed algorithms. Day 28 closed with a direct forward reference to this day: once it becomes clear that a neural network can learn a better keypoint detector and descriptor than any hand-designed one (Day 28 §7), the natural next question is how such a network actually works. This day answers that question from first principles.

1.1 Day 28's Closing Thread: SIFT/ORB/HOG as Handcrafted Invariances

It is worth recalling precisely what "handcrafted" meant across Days 26 through 28, since it sharpens exactly what changes in this day. SIFT's 128-dimensional descriptor (Day 28 §3.1.1) was built from a human-chosen recipe: compute gradient orientation histograms within a 4×4 grid of subregions, normalise to unit length, done. ORB's binary descriptor (Day 28 §3.3) was built from a different human-chosen recipe: sample 256 fixed pixel-pair comparisons, done. HOG (Histogram of Oriented Gradients, referenced in Day 26 §5.3) follows the same pattern for a different task: compute gradient orientation histograms over a sliding window, done. In every case, a human researcher decided, in advance, which mathematical operations would produce a useful representation, and validated that choice empirically after the fact — Day 28 §7.1 identified this as the reason classical descriptors hit a ceiling under extreme conditions they were never specifically designed to handle.

It is worth being precise about what actually changes and what does not, since it is easy to overstate the shift. The general pipeline structure — extract local structure, build a representation, use that representation for a downstream task — remains essentially the same across both eras; Day 28's detect-describe-match-filter-estimate pipeline (Day 28 §1.2) and a CNN's convolution-pooling-activation-classification pipeline (this day's §6.1) share the same basic shape of "extract structure, then use it." What changes is entirely how the extraction stage's parameters are determined: fixed by a human in the classical era, discovered by gradient descent in the deep learning era. Recognising this continuity, rather than treating deep learning as an unrelated, unprecedented departure, makes the transition this day covers considerably easier to internalise.

1.2 The 2012 AlexNet Moment: A Recap

Day 26 §6.1 introduced the 2012 ImageNet result in brief: a deep convolutional network called AlexNet reduced the best classical approach's top-5 error rate from roughly 26% to roughly 15%, a step-change large enough to redirect the field's research effort away from hand-engineered features entirely. This day exists to explain, in full technical detail, exactly what a convolutional neural network is and how it manages to learn representations that outperform decades of careful, expert hand-engineering — the mechanism Day 26 introduced only at the level of "this happened and it mattered," which this day now unpacks completely.

1.3 Why This Shift Happened: Data, Compute, and Architecture

Day 26 §1.1 already previewed the three ingredients behind the 2012 breakthrough — a sufficiently large labelled dataset (ImageNet), sufficiently powerful and affordable parallel compute (GPUs), and a handful of training tricks that made deep networks tractable to train at all. This day adds a fourth, equally important ingredient that Day 26 deliberately deferred: architectural insight — specifically, the convolution operation itself (Section 2), and the reasons it suits image data so naturally (a topic Day 26 §6.2 introduced at a conceptual level and this day now develops mathematically). Understanding all four ingredients together — data, compute, training tricks, and architecture — is necessary to understand why 2012 was the moment this shift happened, rather than 1989 (when LeCun's conceptually similar LeNet was first published) or some other year entirely.

Ingredient	What Changed by 2012	Covered In
Data	ImageNet: 14M+ labelled images, 20,000+ categories	Day 26 §1.1
Compute	GPUs repurposed for general-purpose parallel computation	Day 26 §1.1
Training tricks	ReLU activations, dropout regularisation	§5.2, §8.2 (this day)
Architecture	Deep stacks of learned convolutional filters	§2–§6 (this day)

Side-by-Side: A Hand-Designed Filter vs. a Learned One

A Sobel kernel (Day 26 §5.1) is a fixed 3×3 matrix, chosen by a human to approximate a horizontal or vertical intensity gradient — its nine weights never change, regardless of what images it is applied to.

A single learned convolutional filter (Section 2 of this day) starts as random noise and is updated by gradient descent (Section 7) until its nine (or more) weights minimise a training loss — and empirically, many early-layer learned filters end up looking remarkably similar to hand-designed edge detectors, discovered independently from data rather than from human insight. This convergence is itself a striking piece of evidence that Sobel-style edge detection is a genuinely useful computation, not merely a human's arbitrary preference.

Year	Milestone	Significance
1980	Fukushima's Neocognitron	First hierarchical, convolution-like visual architecture
1989–1998	LeCun's LeNet	First practical, commercially deployed CNN (§6.2)
2009	ImageNet dataset released	Provided the large labelled dataset CNNs needed to scale (§1.3)
2012	AlexNet wins ImageNet challenge	The step-change moment that redirected the field (§1.2)
2014	Dropout published	Regularisation technique enabling deeper, less overfit networks (§8.2)
2015	Batch normalization published	Training-stability technique enabling faster, deeper training (§8.3)

2 The Convolution Operation

2.1 Convolution as a Sliding-Window Weighted Sum

The convolution operation, illustrated concretely in Figure 1, is mechanically identical to the filtering operations Day 26 §5 and Day 27 §4 already introduced: a small kernel (a matrix of weights) slides across the input, and at each position, the kernel's weights are multiplied element-wise against the input values underneath it, and the results are summed to produce a single output value at that position. Repeating this at every valid position across the input produces a complete output feature map.

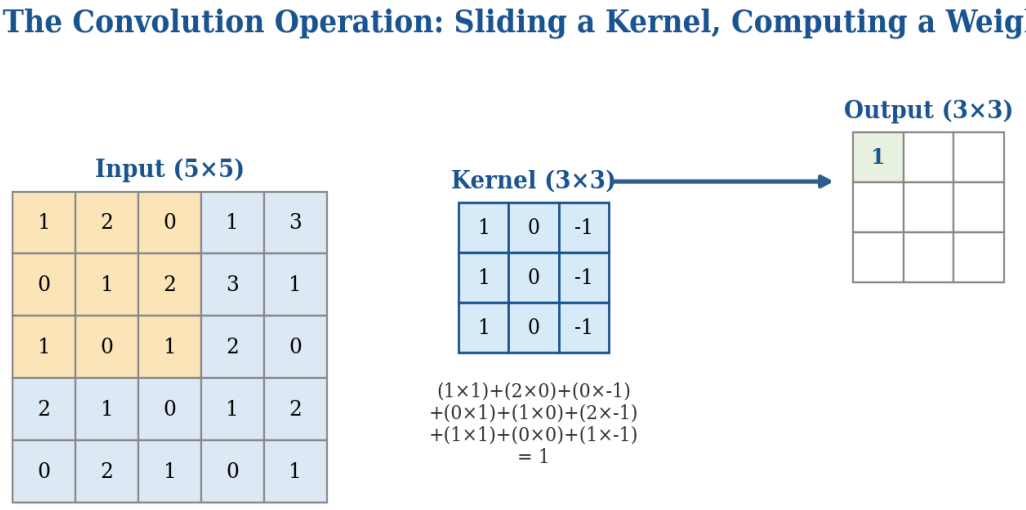

Figure 1. *The convolution operation: a 3×3 kernel slides across a 5×5 input, computing a weighted sum at each position to produce a 3×3 output.*

Worked Example: Computing One Output Value by Hand

From Figure 1: the highlighted 3×3 input patch is [[1,2,0],[0,1,2],[1,0,1]], and the kernel is [[1,0,-1],[1,0,-1],[1,0,-1]].

Element-wise multiply and sum: (1×1)+(2×0)+(0×−1) + (0×1)+(1×0)+(2×−1) + (1×1)+(0×0)+(1×−1) = 1+0+0 + 0+0−2 + 1+0−1 = 1.

This single number becomes the top-left value of the output feature map. Sliding the kernel one position to the right (or down, depending on the scan order) and repeating this computation produces the next output value, and so on across the entire valid input area — exactly the process that produces the full 3×3 output shown in Figure 1.

2.2 Cross-Correlation vs. True Convolution

A mathematical technicality is worth flagging explicitly, since it trips up anyone who has studied signal processing before encountering deep learning. True mathematical convolution, as defined in signal processing, flips the kernel both horizontally and vertically before sliding it across the input. What deep learning frameworks (and this day's Figure 1) actually compute is cross-correlation — sliding the kernel across the input without flipping it first. The two operations are mathematically distinct, but the distinction is immaterial for a learned kernel: since the kernel's weights are learned via gradient descent (Section 7) rather than hand-derived from signal-processing theory, a network trained with cross-correlation simply learns the flipped version of whatever kernel it would have learned under true convolution — the end-to-end behaviour of the trained network is identical either way. The deep learning field has, by convention, kept the name "convolution" for what is technically cross-correlation, and this compendium follows that convention throughout.

This convention is worth remembering specifically because it can cause confusion when reading classical signal-processing literature (Day 26 and Day 27's filtering content among it) alongside deep learning literature: Day 26 §5.1's Sobel kernels and Day 27 §4.2's Gaussian blur kernels are typically applied via true correlation in OpenCV's implementation as well, for exactly the same practical reason — the distinction only matters when a specific, hand-derived kernel's exact orientation is significant, which is rarely the case for the symmetric or near-symmetric kernels common in both classical image filtering and learned CNN layers alike.

2.3 Kernels as Learnable Parameters

The single most important conceptual shift this section introduces, relative to every filtering operation covered in Days 26 and 27, is this: a convolutional kernel's weights are not fixed by a human designer, as Sobel's or Gaussian blur's were — they are learnable parameters, initialised randomly and updated by the training process (Section 7) to minimise a loss function. A single convolutional layer typically contains many such kernels (Section 2.5), each free to specialise toward detecting whatever pattern turns out to be useful for the task at hand, discovered from data rather than designed by hand.

2.4 Why Local Receptive Fields and Weight Sharing Suit Images

Day 26 §6.2 introduced, at a conceptual level, the three structural properties that make convolutions well-suited to image data: local receptive fields, weight sharing, and the translation invariance this combination produces. Figure 2 makes the first two of these properties visually concrete by contrasting a convolutional layer against a fully-connected layer applied to the same input.

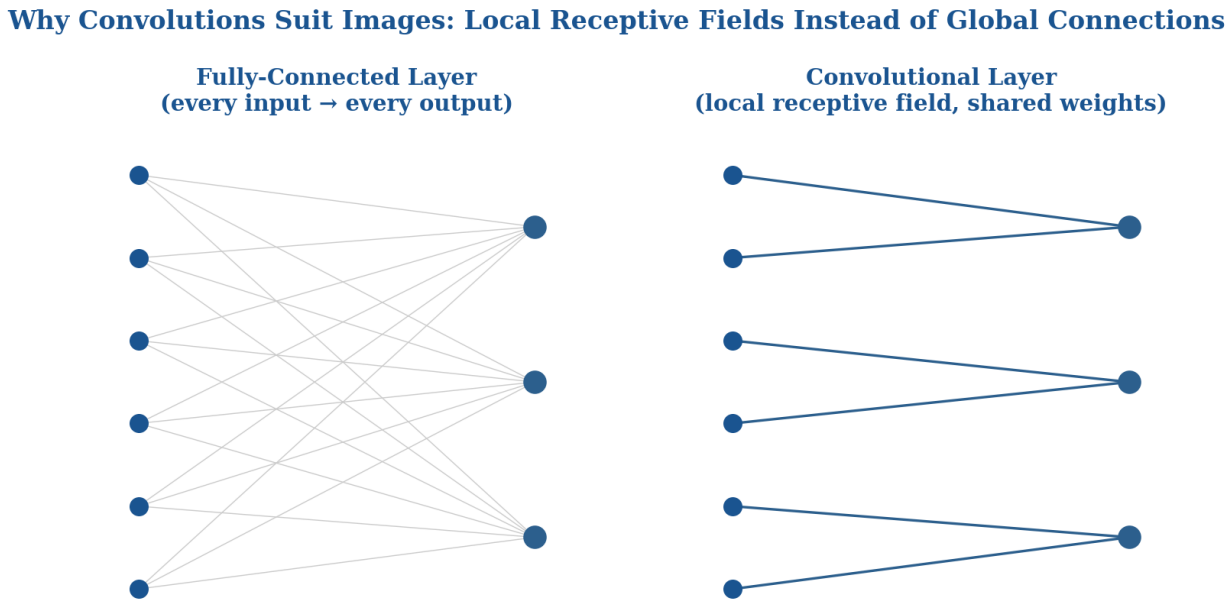

Figure 2. *A fully-connected layer connects every input to every output; a convolutional layer connects each output only to a small local neighbourhood, with the same weights reused at every position.*

Worked Example: Parameter Count: A Recap and Extension of Day 26 §6.2

Day 26 §6.2 computed that a 224×224×3 input connected via a fully-connected layer to 64 outputs requires roughly 9.6 million weights, versus roughly 4,800 weights for an equivalent 5×5 convolutional layer — a difference of roughly 2,000×.

This day extends that comparison with the connection-count intuition Figure 2 illustrates directly: in the fully-connected panel, every single output unit has a distinct connection (and therefore a distinct learnable weight) to every input unit. In the convolutional panel, every output unit connects to only a small local neighbourhood of inputs, AND — critically — every output unit reuses the exact same set of weights, just applied to a different input neighbourhood.

It is this second property, weight sharing, that does most of the parameter-reduction work: even a convolutional layer with a fairly large local receptive field would still be far cheaper than a fully-connected layer, purely because the same small set of weights is reused at every spatial position rather than learned independently for each one.

2.5 Connection to Day 27: Learned Kernels, Same Underlying Operation

It is worth stating explicitly a connection Day 27 §4.1 already flagged in the opposite direction: every filtering operation covered in that day — Gaussian blur, median filtering, sharpening — computes the identical sliding-window weighted-sum operation this section has just formalised, with the sole difference being where the kernel's weights come from. A Gaussian blur kernel's weights are derived analytically from the Gaussian probability density function; a convolutional layer's kernel weights are learned via gradient descent. This is not a superficial analogy — it is the exact same mathematical operation, and recognising this is what makes it possible to understand a CNN's early layers, once trained, by directly

inspecting their learned kernels and comparing them against the classical kernels from Day 26 and Day 27, a comparison Section 9 returns to in depth.

2.6 Bias Terms

Every convolutional filter this section has discussed also carries an associated bias term — a single learnable scalar added to the result of the weighted-sum computation at every spatial position, after the kernel's multiply-and-sum step but before any activation function (Section 5) is applied. The bias allows a filter to shift its activation threshold independently of its weight pattern: two filters with identical kernel weights but different bias values will activate at different overall input intensities, giving the network an extra degree of freedom per filter beyond what the kernel weights alone provide. Bias terms are a small addition to a layer's total parameter count (one value per filter, regardless of kernel size or input channel depth, as reflected already in Section 3.4's worked examples), but they are rarely omitted in practice, and their omission is occasionally used deliberately in specific architectural patterns — most notably immediately before a batch normalization layer (Section 8.3), since batch normalization's own learned shift parameter makes an immediately preceding bias term redundant.

3 Convolutional Layer Mechanics

3.1 Stride

Stride determines how far the kernel moves between successive applications — a stride of 1 (the default assumed in Section 2's examples) slides the kernel one position at a time, producing an output nearly as large as the input; a stride of 2 skips every other position, producing an output roughly half the input's size in each spatial dimension. Figure 3's third panel illustrates stride 2 directly. Larger strides reduce computational cost and output resolution simultaneously, and are sometimes used as a direct alternative to the pooling operations covered in Section 4.

Stride and Padding Control the Output Feature Map Size

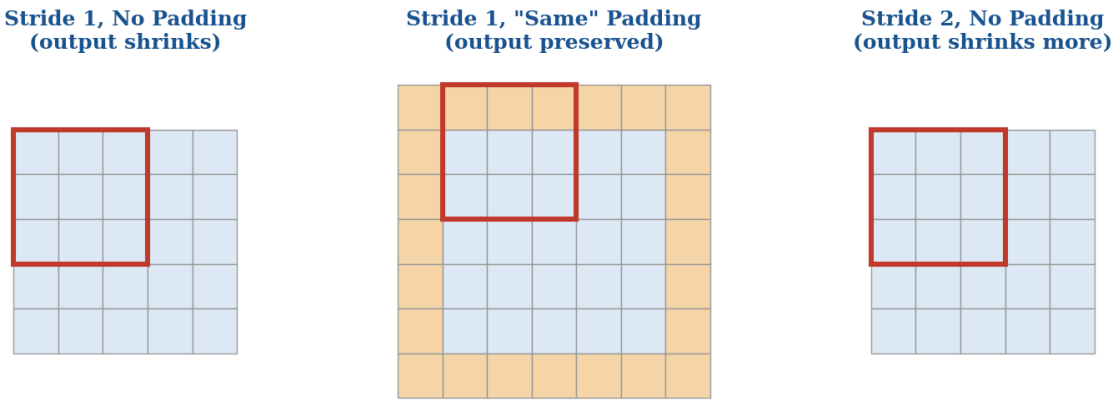

Figure 3. Stride and padding both influence the output feature map's size: larger strides skip positions; padding adds a border around the input to control how much the output shrinks.

3.2 Padding: Valid vs. Same

Without padding ("valid" convolution, Figure 3's first panel), the output feature map is necessarily smaller than the input, since the kernel cannot be centred on a border pixel without extending past the input's edge — for a K×K kernel applied with stride 1, each spatial dimension shrinks by (K−1) pixels. "Same" padding (Figure 3's second panel) adds a border of zeros (or, less commonly, reflected or replicated pixel values) around the input before convolving, sized specifically so the output matches the input's spatial dimensions exactly — a common and convenient choice when a network needs to stack many convolutional layers without the spatial dimensions shrinking away to nothing before the network is deep enough to be useful.

3.3 Output Size & Receptive Field Calculations

The output spatial dimension of a convolutional layer follows directly from the input size, kernel size, stride, and padding: output_size = floor((input_size + 2×padding − kernel_size) / stride) + 1. This formula is worth being able to apply directly, since getting it wrong is a common source of shape-mismatch errors when building a network by hand.

Worked Example: Applying the Output Size Formula

A 32×32 input, 5×5 kernel, stride 1, no padding (valid convolution): output = floor((32 + 0 − 5) / 1) + 1 = 28. Result: 28×28, matching the first convolutional stage in Figure 8's LeNet-style architecture.

The same 32×32 input with "same" padding (padding = 2 for a 5×5 kernel at stride 1): output = floor((32 + 4 − 5) / 1) + 1 = 32. Result: 32×32 — spatial size exactly preserved, as §3.2 described.

A 28×28 input, 5×5 kernel, stride 2, no padding: output = floor((28 + 0 − 5) / 2) + 1 = floor(11.5) + 1 = 12. Result: 12×12 — illustrating that stride, not just padding, directly governs output size.

Receptive field — the region of the original input image that a given unit's output value is ultimately sensitive to — grows with network depth in a way worth understanding explicitly, since it directly determines what scale of pattern a given layer can possibly detect. A single unit in the first convolutional layer of a network built from 3×3 kernels has a receptive field of exactly 3×3 input pixels — it can only "see" that small a patch of the original image, no matter how the rest of the network is structured. A unit in the second such layer, however, has an effective receptive field of 5×5 (each of the 3×3 units it depends on in the first layer themselves each see a 3×3 patch, and these patches overlap and extend the combined coverage), and this growth compounds with every additional layer — which is precisely why deep networks (Day 30's subject) are able to recognise large-scale patterns (whole objects) despite being built entirely from small, local kernels: depth, not kernel size alone, is what ultimately grows the receptive field large enough to span an entire object.

3.4 Multi-Channel Convolution and 1×1 Convolutions

Every worked example so far has implicitly assumed a single-channel input, but real images (Day 26 §2.1) have multiple channels — typically 3 for RGB — and intermediate feature maps within a CNN typically

have many more. Figure 4 shows how convolution generalises to this multi-channel case: a single filter's depth always matches the input's channel count exactly (a filter applied to a 3-channel RGB input is itself 3 channels deep, computing a sum across all three channels simultaneously at each spatial position, collapsing them into a single output value), while the number of distinct filters in the layer determines the output's channel count — N filters produce an N-channel output, regardless of how many channels the input had.

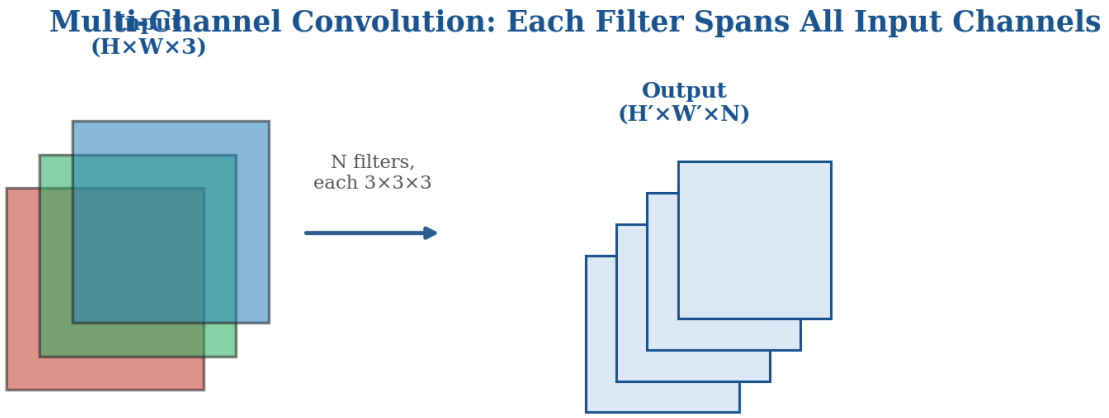

Figure 4. *Multi-channel convolution: each filter spans the full depth of the input, and the number of filters determines the output's channel count.*

1×1 convolutions — a kernel spanning only a single spatial position but the full input channel depth — might initially seem pointless, since they compute no spatial mixing at all. Their actual purpose is channel-wise mixing and dimensionality control: a 1×1 convolution with fewer output filters than input channels compresses the channel dimension (a cheap way to reduce the computational cost of subsequent layers), while a 1×1 convolution with more output filters than input channels expands it, and even a 1×1 convolution with the same number of filters as input channels still performs a learned linear recombination of the channels at every spatial position, followed by a non-linearity (Section 5) if one is applied. This technique becomes especially important in Day 30's more sophisticated architectures, where 1×1 convolutions are used extensively to control computational cost within deeper networks.

Worked Example: Parameter Count for a Multi-Channel Layer

A convolutional layer with 64 filters, each 5×5×3 (matching a 3-channel RGB input), has 5×5×3×64 = 4,800 weights, plus 64 bias terms (one per filter) = 4,864 total learnable parameters — regardless of the input image's spatial resolution, since weight sharing (§2.4) means the same 4,864 parameters are reused at every spatial position.

Compare this to a 1×1 convolution reducing those same 64 channels down to 16: 1×1×64×16 = 1,024 weights plus 16 biases = 1,040 parameters — a cheap operation specifically because the kernel has no spatial extent at all, only channel-mixing weights.

3.5 Dilated (Atrous) Convolutions

Dilated convolutions insert gaps between a kernel's weights, effectively increasing the receptive field a kernel covers without increasing its parameter count or computational cost — a 3×3 kernel with a dilation rate of 2, for instance, spans a 5×5 region of the input but still only has 9 learnable weights, sampling every other input position rather than every position contiguously. This is a genuinely useful technique for tasks that need a large receptive field without the parameter and compute cost of a correspondingly large ordinary kernel, and it is previewed here specifically because it becomes central to Day 33's treatment of semantic segmentation, where preserving fine spatial resolution while still achieving a large receptive field is a core architectural tension.

Worked Example: Comparing Receptive Field Growth: Standard vs. Dilated

A 3×3 kernel with dilation rate 1 (standard convolution) covers exactly a 3×3 receptive field, using 9 weights.

The same 3×3 kernel with dilation rate 2 covers a 5×5 receptive field — more than double the spatial coverage — while still using only 9 weights, since the gaps between sampled positions contribute no additional parameters.

A 3×3 kernel with dilation rate 4 covers a 9×9 receptive field with, again, only 9 weights — illustrating how dilation lets receptive field grow rapidly with dilation rate while parameter count stays completely flat, a very different trade-off from simply using a larger ordinary kernel (a 9×9 standard kernel achieving the same 9×9 receptive field would require 81 weights, 9× more).

4 Pooling & Spatial Downsampling

4.1 Max Pooling vs. Average Pooling

Pooling operations reduce a feature map's spatial dimensions by summarising each small local region (typically 2×2) with a single value, illustrated concretely in Figure 5. Max pooling takes the maximum value within each pooling window, retaining the strongest activation in each local region and discarding the rest — the standard default in most CNN architectures, motivated by the intuition that the strongest response to a learned filter within a small region is usually the most informative signal, and small positional shifts within that region should not change which feature was detected, only possibly its exact reported location. Average pooling takes the mean value within each window instead, retaining a smoothed summary of the entire region rather than just its peak.

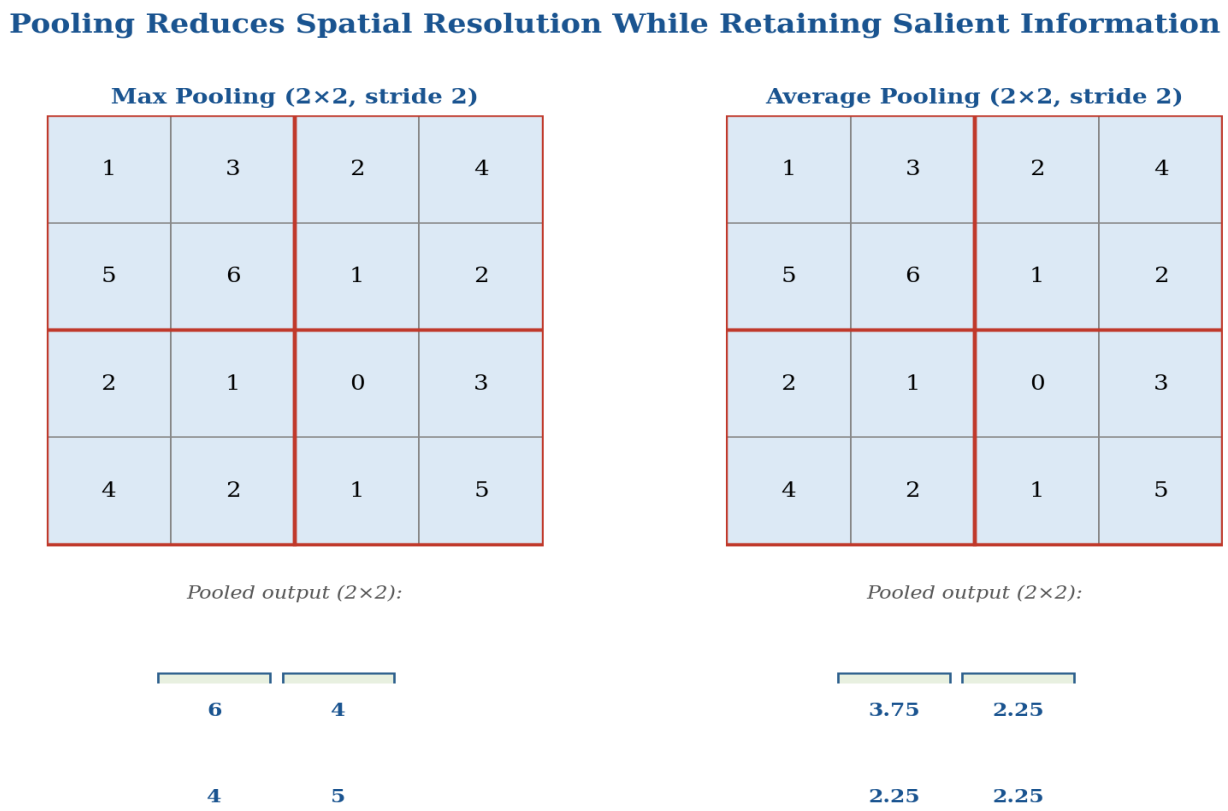

Figure 5. *Max pooling retains the strongest activation in each local region; average pooling retains the mean. Both reduce a 4×4 input to a 2×2 output.*

4.2 Purpose: Translation Invariance, Dimensionality Reduction, and Efficiency

Pooling serves three related purposes worth distinguishing explicitly. Translation invariance is strengthened beyond what convolution's weight sharing alone provides (Section 2.4): because max pooling reports only the strongest activation within a window regardless of its exact position inside that window, a feature detected at slightly different positions across two similar images still produces the same pooled output, making the network more robust to small spatial shifts in exactly where a detected feature happens to fall. Dimensionality reduction follows directly from pooling's spatial downsampling — a 2×2 pooling operation with stride 2 halves both spatial dimensions, reducing a feature map's total size by a factor of 4, which correspondingly reduces the computational cost of every subsequent layer. Computational efficiency is the direct consequence of this dimensionality reduction: fewer spatial positions means fewer convolution operations required in later layers, letting a network afford to increase its channel depth (Section 3.4) as it goes deeper, without total computational cost spiralling out of control — exactly the "channel depth increases while spatial dimensions decrease" pattern Section 6 identifies as the standard architectural template.

Worked Example: Compute Savings from a Single Pooling Stage

Consider a 224×224×64 feature map, followed by a 2×2 max pooling stage (stride 2), then a 3×3×128 convolutional layer.

Without pooling: the 3×3×128 conv layer operates on a 224×224 spatial grid — 224 × 224 = 50,176 spatial positions, each requiring a 3×3×64×128 multiply-accumulate operation.

With pooling: the same conv layer now operates on a 112×112 grid — 112 × 112 = 12,544 positions, a 4× reduction in spatial positions and therefore roughly a 4× reduction in that layer's total computational cost.

This 4× saving compounds with every additional pooling stage in the network, which is precisely why §6.4's "spatial dimensions shrink while channel depth grows" pattern is computationally affordable rather than prohibitively expensive — pooling's savings are what fund the later layers' increased channel depth.

4.3 Global Average Pooling

Global average pooling (GAP) takes averaging to its logical extreme: rather than pooling over small local windows, it averages an entire feature map — of whatever spatial size it happens to be — down to a single value per channel. Applied as the final operation before a classification head, GAP eliminates the need for the large flatten-and-fully-connect step Section 6's classic architecture uses, dramatically reducing the parameter count of the network's final layers (a fully-connected layer connecting a 7×7×512 feature map to even a modest 1,000-unit output would require over 25 million weights; GAP followed by a much smaller fully-connected layer from 512 to 1,000 requires only about 512,000). GAP has become the standard choice in most modern architectures (previewed here ahead of Day 30's detailed coverage of ResNet and its successors) precisely because of this parameter efficiency, along with a useful side benefit: a GAP-based network can accept input images of varying spatial resolution, since global averaging collapses whatever spatial size the final feature map happens to have down to the same fixed per-channel output regardless.

4.4 Strided Convolutions as a Learned Alternative to Pooling

It is worth noting, as a bridge to Day 30's more advanced architectures, that pooling is not the only way to achieve spatial downsampling — Section 3.1's strided convolution accomplishes the same dimensionality reduction, but with the downsampling operation itself being learned rather than a fixed max or average rule. Some modern architectures replace pooling layers entirely with strided convolutions on the reasoning that a learned downsampling operation can, in principle, discover a better way to summarise a local region than a fixed max or average rule ever could — a genuine architectural choice with trade-offs on both sides (a strided convolution adds learnable parameters and computational cost that pooling does not have; pooling contributes no parameters at all and imposes a specific, well-understood inductive bias) that recurs throughout the architecture-design decisions Day 30 covers in depth.

A final practical note worth adding here: the choice between pooling and strided convolution is rarely an all-or-nothing decision within a single architecture. Many practical networks mix both — using pooling in some stages, where its parameter-free, well-understood inductive bias is preferred, and strided convolutions in others, where the additional flexibility of a learned downsampling rule is judged worth its extra parameter and compute cost. There is no universally correct choice; it is an empirical, architecture-specific decision

revisited throughout Day 30's survey of how these design choices played out across the field's major architectures.

Downsampling Method	Learnable?	Adds Parameters?	Inductive Bias
Max pooling	No	No	Strongest local activation matters most
Average pooling	No	No	Overall local signal matters, not just the peak
Strided convolution	Yes	Yes	None imposed — network decides what matters

5 Activation Functions & Non-Linearity

5.1 Why Non-Linearity Is Necessary

Every operation covered so far in this day — convolution (Section 2) and pooling (Section 4) — is either linear or, in max pooling's case, piecewise simple in a way that does not add genuine representational power on its own. This matters because stacking multiple linear operations collapses mathematically into a single equivalent linear operation: if layer 1 computes $y = W_1x$ and layer 2 computes $z = W_2y$, then $z = W_2(W_1x) = (W_2W_1)x$ — mathematically indistinguishable from a single linear layer with weights W_2W_1, no matter how many linear layers are stacked. Without a non-linear function inserted between layers, a deep stack of convolutional layers would have no more representational capacity than a single layer, regardless of how many layers or how many parameters the network contains — non-linearity is what actually allows depth to matter.

5.2 ReLU and Its Dominance

ReLU (Rectified Linear Unit), defined simply as $f(x) = \max(0, x)$ and plotted in Figure 6, is the dominant activation function in modern CNNs, and understanding why requires contrasting it against the sigmoid and tanh functions that were the historical default before ReLU's widespread adoption (itself directly credited as a contributing factor in AlexNet's 2012 success, per Section 1.2). Sigmoid and tanh both saturate — their output approaches a fixed asymptote (0 or 1 for sigmoid; −1 or 1 for tanh) for large positive or negative inputs, and in the saturated region, their gradient (the slope of the function) approaches zero. This is directly damaging for training a deep network via backpropagation (Section 7.1), since gradients from later layers are multiplied together as they propagate backward through earlier layers — if several layers' activations are saturated simultaneously, the accumulated gradient can shrink toward zero so severely that earlier layers receive essentially no training signal at all, a phenomenon called the vanishing gradient problem, previewed here ahead of its central role in motivating Day 30's residual connections.

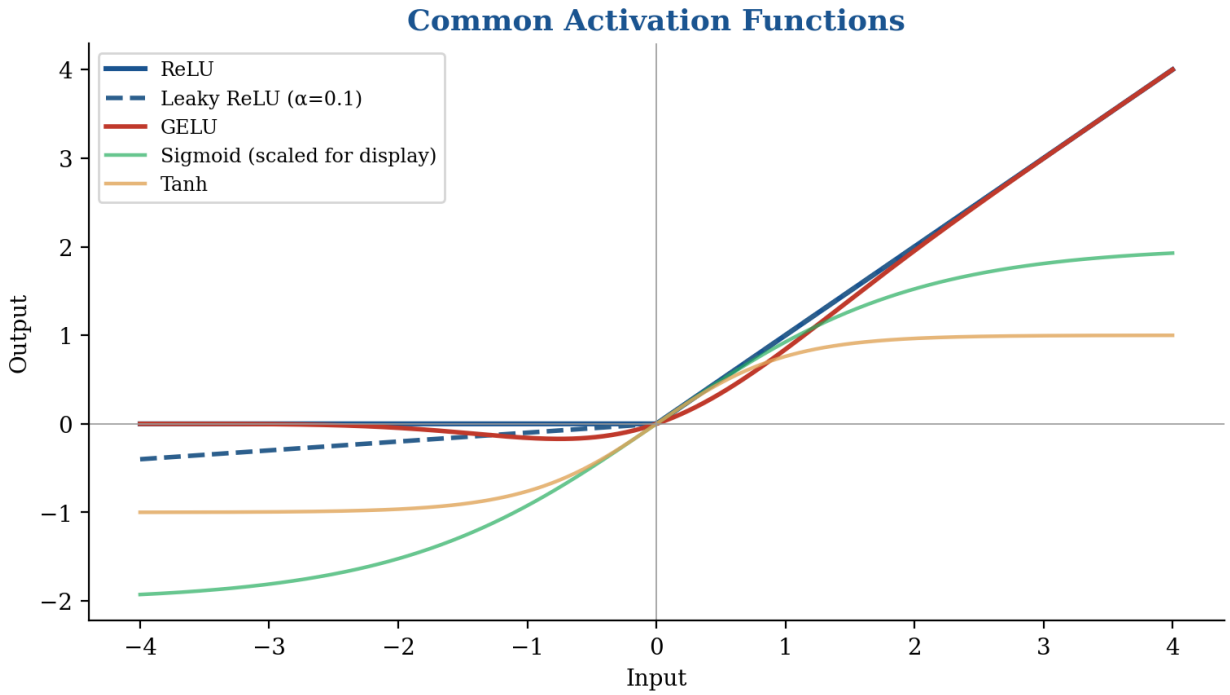

Figure 6. *Common activation functions: ReLU and its variants avoid the saturation that limits sigmoid and tanh, particularly for positive inputs.*

ReLU avoids this saturation problem for positive inputs entirely — its gradient is exactly 1 for any positive input, no matter how large, meaning gradients pass through unchanged rather than shrinking. It is also computationally trivial to compute (a single comparison against zero, versus sigmoid and tanh's exponential function evaluations), a meaningful practical advantage when a layer is applied millions of times across a large network and a large training set.

Worked Example: Comparing Gradient Magnitudes: Saturated vs. Unsaturated

Consider an input value of x = 3, passed through both sigmoid and ReLU.

Sigmoid(3) ≈ 0.953, gradient ≈ 0.045 — already substantially saturated at this modest input magnitude; the gradient flowing backward through this unit is reduced to under 5% of its input scale.

ReLU(3) = 3, gradient = 1 — no reduction at all; the full gradient signal passes through unchanged.

Across a network with many stacked layers, per §5.2's vanishing-gradient discussion, sigmoid's per-layer gradient reduction compounds multiplicatively: even a modest 0.045 reduction per layer becomes $0.045^7 \approx 3\times10^{-11}$ after just seven layers — an essentially zero gradient reaching the network's earliest layers, which is precisely the mechanism behind the vanishing gradient problem and precisely why ReLU's gradient-preserving behaviour for positive inputs matters so much for training genuinely deep networks.

5.3 ReLU Variants: Leaky ReLU, ELU, and GELU

ReLU's one significant weakness is the dying ReLU problem, discussed in full in Section 5.4 below; several variants address it directly. Leaky ReLU (Figure 6's dashed line) modifies ReLU's negative-input behaviour: rather than outputting exactly zero for any negative input, it outputs a small negative value

proportional to the input ($f(x) = x$ for $x > 0$, $f(x) = \alpha x$ for $x \leq 0$, with α typically around 0.01–0.1), ensuring a small but non-zero gradient flows even for negative inputs, directly preventing the dying ReLU failure mode. ELU (Exponential Linear Unit) takes a smoother approach for negative inputs, using a scaled exponential curve rather than Leaky ReLU's straight line, which some research has found improves training dynamics further, at a modest additional computational cost from the exponential evaluation. GELU (Gaussian Error Linear Unit, also plotted in Figure 6) is smoother still — rather than ReLU's sharp corner at zero, GELU transitions gradually, weighting each input by its value under a Gaussian cumulative distribution function; GELU has become the standard choice in transformer architectures specifically (a brief forward reference to Day 30's introduction of Vision Transformers), though it appears in some modern CNN architectures as well.

5.4 The Dying ReLU Problem

Because ReLU's gradient is exactly zero for any negative input, a unit whose weights happen to produce a consistently negative output across the entire training set receives zero gradient on every training step, and — since its weights never update — remains permanently "dead," never activating again for any input, effectively removing that unit's capacity from the network for the rest of training. This can happen due to an unlucky weight initialisation, or due to an overly large learning rate (Section 7.3's subject) causing a large, destabilising weight update that pushes a unit into this permanently-negative regime. In practice, a moderate fraction of dead units is common and rarely catastrophic in a sufficiently large network — there is enough redundant capacity elsewhere that a small number of dead units does not meaningfully hurt overall performance — but a large fraction of dead ReLU units across a network is a genuine warning sign worth diagnosing, and is precisely the practical motivation for Leaky ReLU, ELU, or GELU (Section 5.3) as safer defaults when dead units are observed to be a real problem in a specific training run.

Activation	Formula	Gradient for x<0	Dying-Unit Risk
ReLU	max(0, x)	Exactly 0	Highest
Leaky ReLU	x if x>0 else αx	Small, non-zero (α)	Low
ELU	x if x>0 else α(e^x−1)	Small, non-zero, smooth	Low
GELU	x·Φ(x) (Gaussian CDF)	Small, non-zero, smooth	Low

6 Building a CNN Architecture

6.1 The Classic Conv-Pool-Conv-Pool-Flatten-Dense Pattern

Figure 7 shows a complete, classic CNN architecture in the style of LeNet-5 — the historical starting point for essentially every CNN architecture that followed it, including AlexNet. The pattern is straightforward once every individual component has been introduced: alternating convolutional layers (each typically followed by a ReLU activation, Section 5.2) and pooling layers (Section 4) extract and progressively downsample spatial features, followed by a flatten operation that reshapes the final, small spatial feature

map into a single long vector, followed by one or more fully-connected ("dense") layers that combine the extracted features into a final classification decision.

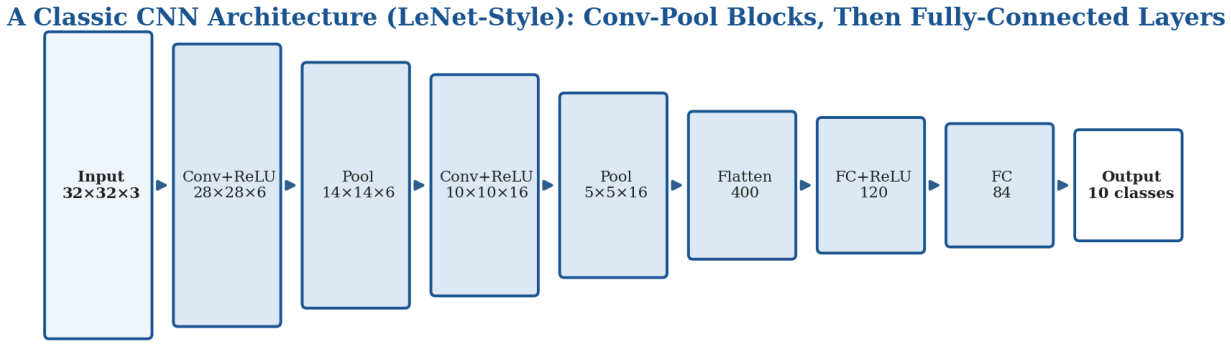

Figure 7. *A classic LeNet-style CNN architecture: alternating convolution and pooling stages extract features, followed by flattening and fully-connected layers for final classification.*

6.2 LeNet-5: The Historical Starting Point

LeNet-5 (LeCun et al., 1989, refined through the 1990s) was designed for handwritten digit recognition and successfully deployed commercially for reading handwritten cheques — a genuinely early, practical deep learning success story that predates the 2012 AlexNet moment (Section 1.2) by over two decades, and a direct illustration of the point Day 26 §1.1 made about architectural ideas often existing well before the data and compute needed to fully exploit them become available. LeNet-5's architecture follows exactly the pattern Figure 7 illustrates: two conv-pool stages followed by fully-connected layers, operating on small 32×32 greyscale digit images — modest by today's standards, but establishing the architectural template that every subsequent CNN, including AlexNet and the far deeper networks Day 30 covers, has built upon.

It is worth appreciating just how narrow LeNet-5's original deployment context was, since it clarifies exactly what "scaling up" (Section 6.3) actually means. LeNet-5 was trained to recognise only 10 classes (the digits 0 through 9) from small, centred, mostly-clean greyscale images — a comparatively constrained problem relative to ImageNet's 1,000 classes of natural photographs at far higher resolution, captured under wildly varying lighting, pose, occlusion, and background clutter. The architectural template survived this jump in problem difficulty essentially unchanged in its basic structure; what had to scale were the network's size (more layers, far more filters per layer), input resolution, and — critically, per Section 1.3 — the volume of training data and compute available to train it, rather than any fundamental redesign of the conv-pool-flatten-dense pattern itself.

6.3 AlexNet: The Modern Inflection

AlexNet (Section 1.2) followed LeNet-5's same fundamental conv-pool-flatten-dense template, scaled up substantially: more layers, far more filters per layer, larger input images (224×224 rather than LeNet's 32×32), and the ReLU activation (Section 5.2) rather than LeNet's original sigmoid/tanh-based activations — a direct illustration of how much of AlexNet's success came from scaling and refining an existing

architectural template (LeNet's) combined with the data, compute, and training-trick improvements Section 1.3 catalogued, rather than a wholly novel architecture invented from scratch.

Property	LeNet-5 (1989/1998)	AlexNet (2012)
Input size	32×32 (greyscale)	224×224 (RGB)
Depth	~5 learnable layers	8 learnable layers
Activation	Sigmoid / Tanh	ReLU (§5.2)
Parameters	~60,000	~60 million
Training data	Thousands of digit images	1.2 million ImageNet images
Hardware	CPU	Multiple GPUs

6.4 Channel Depth Increases While Spatial Dimensions Decrease

A consistent pattern across nearly every classic CNN architecture, visible directly in Figure 7's stage-by-stage dimensions, is worth naming explicitly: as data flows through the network, spatial dimensions (height and width) shrink at each pooling stage, while channel depth grows at each convolutional stage. This is not an arbitrary design choice — it reflects a deliberate trade-off enabled by Section 4.2's pooling-driven dimensionality reduction: because pooling reduces the computational cost of processing each spatial position, the network can afford to detect a much richer set of features (more channels) at each subsequent stage without total computational cost spiralling out of control, and early layers empirically need comparatively few, simple feature types (edges, colours, textures) while later layers benefit from many more, increasingly complex and specific feature types (object parts, and eventually whole objects) — a hierarchy directly visualised in Section 9's filter-visualisation discussion.

Worked Example: Total Parameter Count for a Complete Small CNN

Using Figure 7's architecture: Conv1 (5×5×3×6 + 6 bias) = 456; Conv2 (5×5×6×16 + 16 bias) = 2,416; FC1 (400×120 + 120) = 48,120; FC2 (120×84 + 84) = 10,164; FC3 (84×10 + 10) = 850.

Total: 456 + 2,416 + 48,120 + 10,164 + 850 = 62,006 parameters.

Notice that the fully-connected layers (Section 6.1's "flatten-dense" stage) account for roughly 95% of this small network's total parameters (59,134 of 62,006), despite the convolutional layers doing the bulk of the actual feature-extraction work — a pattern that becomes even more pronounced in larger networks, and directly explains why Section 4.3's Global Average Pooling, which eliminates the large flatten-to-dense transition entirely, produces such a dramatic parameter-count reduction in modern architectures.

6.5 A Forward Reference to Day 30

This section has deliberately stayed close to the classical, historical architecture template established by LeNet and scaled by AlexNet. Day 30 picks up immediately from here, surveying how this basic template evolved into today's much deeper, more sophisticated backbone architectures — VGG's systematic use of small, stacked 3×3 kernels; ResNet's residual connections, developed specifically to overcome the

vanishing gradient problem (Section 5.2) that limits how deep a plain conv-pool-flatten-dense network of the kind covered in this section can practically be trained; and the efficient, mobile-friendly architectures that followed. Everything covered in this day is the necessary foundation for that later, more advanced material — Day 30 assumes fluent, comfortable familiarity with every concept this day has introduced.

```
import torch.nn as nn

# A LeNet-style CNN, directly matching Figure 7's architecture
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 6, kernel_size=5),    # §2-3: learnable kernel
            nn.ReLU(),                          # §5.2: non-linearity
            nn.MaxPool2d(2, stride=2),          # §4.1: downsampling
            nn.Conv2d(6, 16, kernel_size=5),
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),                       # §6.1
            nn.Linear(16 * 5 * 5, 120),
            nn.ReLU(),
            nn.Dropout(0.5),                    # §8.2: regularization
            nn.Linear(120, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

7 Training a CNN

7.1 Backpropagation Through Convolutional Layers

Backpropagation is the algorithm that computes how much each learnable parameter in a network (every convolutional kernel weight, every fully-connected layer weight) contributed to the network's prediction error, working backward from the output layer to the input layer by repeated application of the chain rule of calculus. Figure 8 shows this as part of the complete training loop this section develops. A full mathematical derivation is beyond this day's scope, but the conceptual picture is essential: backpropagation computes the gradient of the loss function (Section 7.2) with respect to every single learnable parameter in the network, and these gradients indicate the direction each parameter should move to reduce the loss — precisely the information the optimiser (Section 7.3) needs to update the network's weights.

It is worth being explicit about why backpropagation through a convolutional layer specifically is efficient, rather than assuming it simply works the same way as backpropagation through a fully-connected layer without modification. Because a convolutional layer's weights are shared across every spatial position (Section 2.4), the gradient with respect to a single kernel weight must be accumulated across every position where that weight was used during the forward pass — summed across the entire spatial extent of the layer's output, rather than computed independently for each position as a fully-connected layer's distinct, unshared

weights would require. This accumulation is what allows a convolutional layer with only a few thousand parameters (per Section 3.4's worked example) to still receive a rich, well-averaged gradient signal from an entire feature map's worth of forward-pass activity, rather than each individual weight only ever seeing the comparatively sparse gradient information a single spatial position could provide on its own.

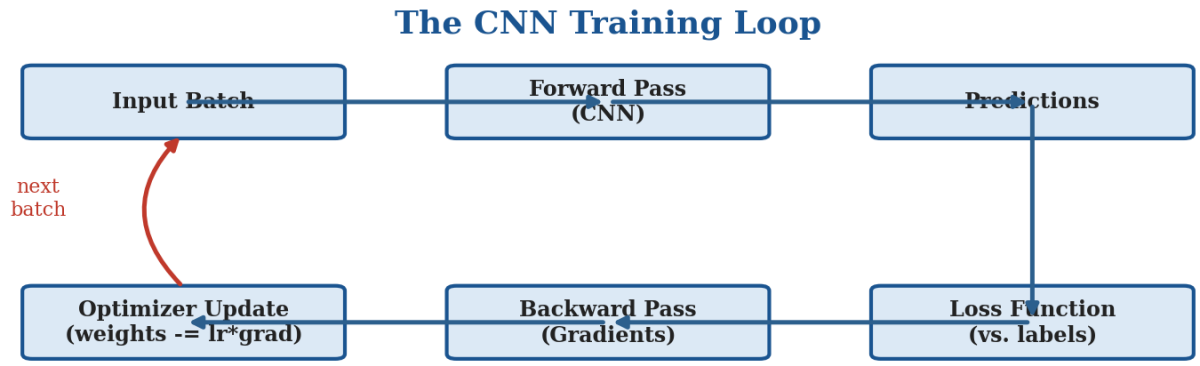

Figure 8. *The complete CNN training loop: forward pass produces predictions, a loss function compares them to true labels, backpropagation computes gradients, and an optimiser updates the weights — repeated across many batches.*

7.2 Loss Functions for Classification: Cross-Entropy

For classification tasks — the primary task this day has used as a running example — cross-entropy loss is the standard choice, measuring the discrepancy between the network's predicted probability distribution over classes (produced by a softmax function applied to the final layer's raw outputs) and the true, correct class label. Cross-entropy loss is small when the network assigns high probability to the correct class and large when it assigns low probability to the correct class, and — critically for training dynamics — its gradient is particularly well-behaved (large when the prediction is confidently wrong, shrinking smoothly as the prediction approaches correctness), which makes it a numerically favourable choice for gradient-based optimisation compared to some alternative loss formulations.

Worked Example: Computing Cross-Entropy Loss for a Single Prediction

Suppose a network predicts class probabilities [0.7, 0.2, 0.1] for a 3-class problem, and the true label is class 0.

Cross-entropy loss = −log(predicted probability of the TRUE class) = −log(0.7) ≈ 0.357 — a fairly small loss, since the network assigned high confidence to the correct class.

Now suppose the network instead predicted [0.1, 0.7, 0.2] for the same true label (class 0):

Loss = −log(0.1) ≈ 2.303 — a much larger loss, correctly penalising the network heavily for confidently predicting the wrong class.

This asymmetry — small loss for confident correct predictions, rapidly growing loss for confident incorrect predictions — is exactly what drives the network's weights toward assigning high probability to correct labels during training via the backpropagation and optimiser steps §7.1 and §7.3 describe.

7.3 Optimizers: SGD with Momentum and Adam

Once gradients are computed via backpropagation, an optimiser determines exactly how to use them to update the network's weights. Stochastic Gradient Descent (SGD) in its simplest form updates each weight by subtracting the learning rate multiplied by that weight's gradient: weight ← weight − (learning_rate × gradient) — directly visible as the final step in Figure 8's training loop. SGD with momentum extends this by accumulating a running average of past gradients, which both smooths out noisy gradient estimates from individual small batches and helps the optimisation process build up speed in directions of consistent improvement, similarly to how physical momentum helps a ball roll consistently downhill rather than oscillating with every small bump. Adam (Adaptive Moment Estimation) goes further, maintaining a separate, adaptively-scaled effective learning rate for every individual parameter in the network, based on that parameter's own recent history of gradient magnitude and variance — a more sophisticated and often faster-converging approach that has become the default optimiser choice for the large majority of modern deep learning training, including most CNN training in practice.

Optimizer	Per-Parameter Adaptive Rate?	Typical Convergence Speed	Common Use
SGD (plain)	No	Slow, but often best final result	Long, carefully-tuned training runs
SGD + Momentum	No	Faster than plain SGD	A strong, widely-used default
Adam	Yes	Fast, especially early in training	Default choice for most modern training

7.4 Learning Rate Scheduling

The learning rate — the single scalar multiplier controlling how large each weight update step is — is widely regarded as the single most consequential hyperparameter in training any neural network, CNNs included. Too high a learning rate causes training to diverge or oscillate wildly rather than converging to a

good solution; too low a learning rate causes training to converge extremely slowly, wasting compute and potentially getting stuck in a poor local solution before ever reaching a good one. Learning rate scheduling — systematically reducing the learning rate over the course of training, often starting comparatively high to make rapid early progress and decreasing toward the end to allow fine, stable convergence — is standard practice in nearly every serious CNN training run, and the specific scheduling strategies (step decay, cosine annealing, and others) parallel closely the learning-rate-scheduling techniques used elsewhere in deep learning more broadly.

Worked Example: Diagnosing Training Instability from the Loss Curve

Learning rate far too high: training loss oscillates wildly from epoch to epoch, sometimes increasing sharply rather than decreasing — each weight update overshoots past a good solution rather than approaching it.

Learning rate far too low: training loss decreases, but so slowly that after many epochs it has barely moved from its initial value — easily mistaken for a bug elsewhere in the pipeline (per §11's debugging checklist) rather than simply an underpowered learning rate.

A well-chosen learning rate with scheduling: loss decreases quickly and fairly smoothly early in training, then continues decreasing more gradually as the schedule reduces the learning rate, producing a characteristic steep-then-flattening curve rather than either the oscillation or the near-flat line above.

7.5 Transfer Learning and Fine-Tuning: A Direct Practical Link

Day 26 §6.3 introduced transfer learning conceptually: rather than training a new network from randomly initialised weights for every new problem, practitioners overwhelmingly start from a network already pretrained on a large, general dataset and fine-tune it on their specific task, using far less labelled data and compute than training from scratch would require. This is precisely the approach this broader research programme's own PP12 practical project (Day 26) uses in practice: a pretrained MobileNetV2 or ResNet backbone, fine-tuned for mask classification, is fine-tuning in exactly the technical sense this section has now fully explained — the pretrained network's weights serve as the starting point (rather than random initialisation) for the same backpropagation-plus-optimiser training loop this section has developed, typically run for comparatively few epochs at a comparatively low learning rate (Section 7.4), since the pretrained weights already encode broadly useful features and need only modest adjustment for the new task rather than being learned from scratch.

```
import torch
import torch.nn as nn
import torch.optim as optim

model = SimpleCNN(num_classes=10)
criterion = nn.CrossEntropyLoss()          # §7.2
optimizer = optim.Adam(model.parameters(), lr=1e-3)  # §7.3
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)  # §7.4

for epoch in range(50):
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)             # forward pass (Figure 8)
        loss = criterion(outputs, labels)   # compute loss
```

```
        loss.backward()                    # backpropagation (§7.1)
        optimizer.step()                   # weight update
    scheduler.step()
```

8 Regularization in CNNs

8.1 Overfitting in the CNN Context

Overfitting occurs when a network learns to fit the specific training examples it has seen — including their noise and idiosyncrasies — rather than learning the general, underlying pattern that will also hold for new, unseen data; a severely overfit network achieves excellent training accuracy while performing poorly on held-out validation or test data. CNNs, with their potentially very large number of learnable parameters (Sections 2–3), are genuinely susceptible to overfitting, particularly when the available labelled training data is limited relative to the network's capacity — precisely the scenario Day 26 §6.3's transfer-learning discussion identified transfer learning as helping to mitigate, since a pretrained network's weights already encode useful, general structure rather than needing to be learned entirely from a small, task-specific dataset.

Use Case: Diagnosing Overfitting from a Training Curve

A team training a CNN on a modest 2,000-image medical dataset observed training accuracy climb steadily to 99% while validation accuracy plateaued around 72% after epoch 15 and slowly declined thereafter — a textbook overfitting signature.

Applying three of Section 8's techniques together — dropout (§8.2), weight decay (§8.4), and stronger data augmentation (§8.5, directly reusing Day 27's techniques) — closed most of the gap, bringing validation accuracy up to 85% without materially changing training accuracy, a realistic illustration of how these regularisation techniques are used together in practice rather than any single one being sufficient alone.

8.2 Dropout

Dropout (Srivastava et al., 2014) is one of the most widely used regularisation techniques in deep learning: during training, a randomly selected subset of units in a layer (illustrated in Figure 9) is temporarily set to zero on each training step, with a new random subset chosen every step — forcing the network to avoid relying too heavily on any single unit or small combination of units, since that unit might be unavailable (dropped) on any given training step. This encourages the network to learn more robust, redundant representations, distributed across many units rather than concentrated in a few, which empirically improves generalisation to unseen data.

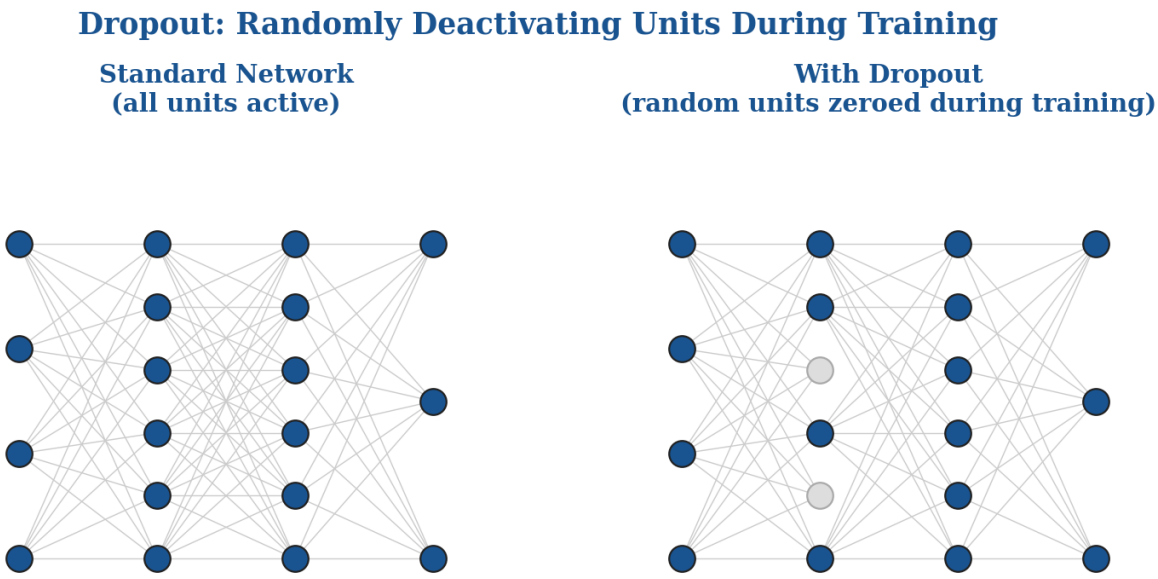

Figure 9. *Dropout randomly deactivates a subset of units during each training step, forcing the network to learn robust, distributed representations rather than depending on any single unit.*

Critically, dropout is applied only during training, exactly analogous to Day 27 §1.2's preprocessing-versus-augmentation distinction: at inference time, all units remain active (no dropping), typically with each unit's output scaled to compensate for the fact that, during training, only a fraction of units were active at any given step on average — a scaling detail most deep learning frameworks handle automatically, but worth understanding conceptually, since forgetting to disable dropout at inference time (leaving units randomly dropped during evaluation) is a genuine, sometimes-encountered bug that produces inconsistent, non-reproducible predictions for the same input, directly analogous to the forgotten-normalization and leftover-augmentation bugs Day 27 §5.2 and §9.3 catalogued.

8.3 Batch Normalization

Day 27 §5.3 previewed batch normalization as a related-but-distinct concept from input normalization, promising full treatment here. Batch normalization is a layer inserted within the network itself — typically after a convolutional layer and before its activation function — that normalises each mini-batch's activations to have zero mean and unit variance (computed across the current training batch), followed by a learned scale and shift that allows the network to undo this normalisation if doing so turns out to be beneficial for a specific layer. Its primary benefit is training stability: by keeping each layer's activation distribution consistent throughout training, batch normalization substantially reduces the sensitivity of training to weight initialisation and learning rate choice, often allowing meaningfully higher learning rates (Section 7.4) to be used safely, which in turn speeds up training convergence considerably. It has a secondary, smaller regularising effect as well, since the batch statistics used for normalisation introduce a small amount of noise into each layer's effective input (since each mini-batch's statistics differ slightly),

though this regularising effect is generally considered secondary to batch normalization's primary training-stability benefit.

A practical detail worth flagging, directly analogous to dropout's train/inference distinction (Section 8.2): batch normalization behaves differently at inference time than during training. During training, the mean and variance used for normalisation are computed fresh from each mini-batch; at inference time, using batch statistics would make a model's prediction for a given input depend on whatever other inputs happen to be in the same inference batch — an undesirable, non-reproducible property for a deployed model. Instead, batch normalization layers track a running average of the mean and variance observed throughout training, and switch to using these fixed, accumulated statistics at inference time — another reason, alongside dropout, that correctly switching a model to evaluation mode (as flagged in this day's §11 debugging checklist) is essential for consistent, reproducible predictions.

8.4 Weight Decay

Weight decay (also known as L2 regularisation) adds a penalty term to the loss function proportional to the sum of the squares of all the network's weights, encouraging the optimiser to prefer smaller weight values unless a larger value is genuinely justified by a meaningful reduction in the primary task loss. The intuition is that very large weights tend to make a network's output highly sensitive to small input changes — a signature of overfitting to noise in the training data — while smaller, more modest weights tend to produce smoother, more generalisable functions. Weight decay is typically controlled by a single hyperparameter scaling the strength of this penalty, and is commonly used together with dropout and batch normalization (Sections 8.2–8.3) as complementary regularisation techniques addressing overfitting from different angles simultaneously.

8.5 Data Augmentation as Regularization: A Direct Callback to Day 27

Day 27 §6 developed data augmentation in full — random crops, flips, colour jitter, and more sophisticated techniques like Cutout, Mixup, and CutMix — as a technique for artificially expanding training data diversity. It is worth being explicit here about exactly why augmentation counts as regularisation in the same technical sense as dropout, batch normalization, and weight decay: by ensuring the network never sees the literal identical pixel values twice across training, augmentation directly discourages the network from memorising specific training examples (the core definition of overfitting, Section 8.1), forcing it instead to learn features robust enough to survive the applied transformations — precisely the same "encourage robustness rather than memorisation" logic underlying every other regularisation technique this section has covered, just implemented at the level of the input data rather than the network architecture or loss function.

Technique	Where Applied	What It Discourages
Dropout (§8.2)	Inside the network (layer activations)	Over-reliance on any single unit
Batch normalization (§8.3)	Inside the network (layer activations)	Sensitivity to activation-scale drift during training

Technique	Where Applied	What It Discourages
Weight decay (§8.4)	The loss function	Unnecessarily large weight values
Data augmentation (§8.5)	The input data	Memorisation of exact training pixel values

9 Visualizing & Interpreting CNNs

9.1 Visualizing Learned Filters

Because a trained convolutional kernel is, mechanically, identical to a classical hand-designed filter (Section 2.5), it can be directly visualised and inspected the same way a Sobel or Gaussian kernel could be — and doing so for a trained network's early-layer filters produces one of deep learning's most striking and widely-reproduced empirical findings. Early-layer filters in a trained CNN, visualised directly as small image patches, overwhelmingly resemble simple, classical edge and colour-blob detectors — strikingly similar to Section 1's callout box comparison against hand-designed Sobel-style kernels, despite being discovered entirely through gradient descent on a training objective with no explicit instruction to detect edges at all. This finding directly echoes Day 26 §3's discussion of Hubel and Wiesel's biological V1 simple cells, which similarly respond to edges and oriented gradients — an independent, non-biological optimisation process (gradient descent on ImageNet classification) converging toward a broadly similar early-stage representation as biological vision, even though, per Day 26 §3.2, the learning mechanisms involved are entirely different.

This convergence between independently-derived hand-designed filters, biological V1 receptive fields, and gradient-descent-discovered CNN filters is worth dwelling on briefly, since it is one of the more philosophically interesting findings this day has covered. Three entirely different processes — decades of human engineering intuition (Sobel and Gabor-style filters), hundreds of millions of years of biological evolution (V1 simple cells), and a few hours or days of gradient descent on a large labelled dataset — converge on broadly the same answer to the question "what is a useful first-stage representation for visual information?" This suggests that edge- and orientation-sensitive filtering is not an arbitrary choice among many equally good options, but something closer to a genuinely necessary, near-optimal solution to the specific statistical structure of natural images — a strong piece of indirect evidence that early-layer CNN representations are discovering something real about the structure of visual data, not merely fitting to arbitrary dataset-specific noise.

9.2 Feature Map Visualization

Beyond visualising the kernels themselves, it is informative to visualise a trained network's feature maps — the actual output produced when a specific image is passed through a specific layer — since this reveals what the network is actually detecting in that specific image, at that specific layer, rather than what a kernel is capable of detecting in the abstract. Early-layer feature maps for a natural photograph typically highlight edges and simple textures, closely matching what direct kernel visualisation (Section 9.1) would predict;

progressively deeper feature maps activate more selectively for larger, more semantically meaningful structures — a partial object part in a mid-depth layer, potentially something recognisable as an entire object-specific pattern in the deepest layers — a direct empirical confirmation of Section 6.4's claim that channel depth and semantic complexity both grow together as data flows deeper into the network.

Worked Example: What a Specific Feature Map Might Encode

Layer 1 (early): a specific channel activates strongly along horizontal edges anywhere in the image — directly comparable to what a Sobel horizontal-gradient kernel (Day 26 §5.1) would produce, but discovered rather than designed.

Layer 4 (middle): a specific channel activates strongly on roughly circular, textured regions — potentially a learned "eye-like" or "wheel-like" detector, combining several Layer 1–3 edge and texture responses into a more complex, spatially larger pattern via the growing receptive field §3.3 described.

Layer 8 (late, near the classification head): a specific channel activates only when a nearly-complete, recognisable object part is present — the kind of highly class-specific response that Section 9.3's Grad-CAM technique directly exploits to localise which image regions drove a final prediction.

This progression — simple and generic, to complex and class-specific — is not designed into the network anywhere explicitly; it emerges from the interaction of §3.3's growing receptive field and §7's gradient-based training pressure to be useful for the final classification objective.

9.3 Class Activation Mapping: Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping, Selvaraju et al., 2017) is the standard technique for visualising which regions of a specific input image most strongly influenced a trained network's prediction for a specific class — producing a heatmap overlay, illustrated in Figure 10, that highlights the image regions a classification decision was actually based on. Mechanically, Grad-CAM computes the gradient of the target class's predicted score with respect to a chosen convolutional layer's feature maps (typically the last convolutional layer, which retains meaningful spatial structure while also encoding the deep, class-discriminative features Section 9.2 described), uses these gradients to weight each feature map channel by its importance to the target class, and combines the weighted feature maps into a single coarse localisation heatmap.

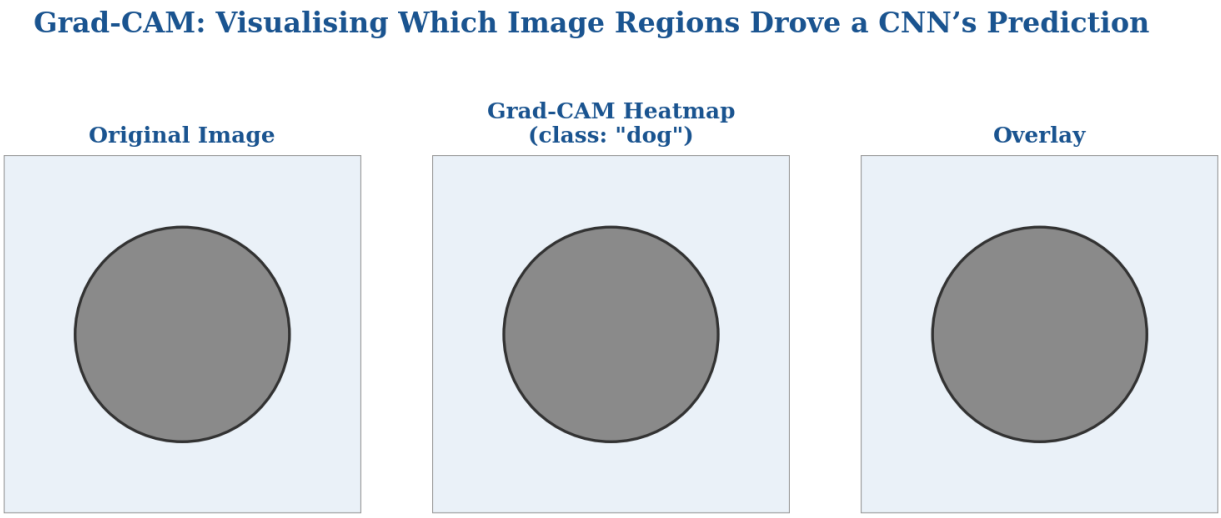

Figure 10. *Grad-CAM produces a heatmap highlighting which regions of an image most influenced a CNN's prediction for a specific class — a direct interpretability tool for an otherwise opaque model.*

Grad-CAM's practical value extends well beyond satisfying curiosity about what a network is "looking at" — it is a genuine debugging and trust-building tool, since a network that achieves high accuracy for the wrong reasons (relying on a spurious background cue correlated with the correct label in the training set, rather than the actual object of interest) can often be caught precisely by noticing that its Grad-CAM heatmap highlights an implausible image region rather than the object itself. This interpretability role becomes centrally important, rather than merely a nice-to-have diagnostic, in Day 37's treatment of medical imaging, where understanding why a model made a specific diagnostic prediction is often a genuine clinical and regulatory requirement, not an optional extra.

It is worth noting a genuine limitation of Grad-CAM, in the same spirit of honest caveat-giving Day 26 §3.2 applied to adversarial examples: the coarse localisation heatmap Grad-CAM produces is derived from the spatial resolution of whichever convolutional layer it is computed from — typically the last convolutional layer before the classification head, which by that point in the network (per Section 6.4's "spatial dimensions shrink with depth" pattern) has a fairly low spatial resolution, often as small as 7×7 for a 224×224 input. Grad-CAM's heatmap is therefore always a coarse, blurry approximation of "which region mattered," upsampled back to the original image resolution for visualisation purposes — it can reliably indicate roughly which part of an image drove a prediction, but should not be over-interpreted as pinpointing an exact pixel-level boundary the way a segmentation mask (Day 33) would.

> ⚠ **When CNNs See What Isn't There: A Deployment Failure Mode**
>
> *A widely cited cautionary example: an image classifier trained to distinguish wolves from huskies achieved excellent validation accuracy, but Grad-CAM-style analysis revealed it was in fact detecting the presence of snow in the background — nearly all of the training set's wolf photographs happened to have snowy backgrounds, and the classifier had learned "snow present" as a highly predictive, spuriously correlated shortcut rather than learning any feature of the animal itself.*
>
> *Deployed against a photograph of a husky standing in snow, or a wolf photographed without snow, this classifier fails confidently and silently — precisely the kind of failure mode that raw accuracy metrics alone cannot detect, but that a Grad-CAM heatmap reveals immediately, connecting directly to §3.2's adversarial-examples discussion of confident, non-human CNN failure modes, and underscoring why interpretability tooling matters for any deployment where training data quality cannot be fully guaranteed.*

9.4 A Forward Reference to Day 37

This section has introduced Grad-CAM as a general-purpose interpretability tool applicable to any trained CNN classifier. Day 37, this module's dedicated explainability and medical imaging day, returns to Grad-CAM (among other, more advanced interpretability techniques) in considerably greater depth, applying it specifically to the high-stakes context of medical diagnosis, where the gap between "the model is usually right" and "a clinician can trust and act on this specific prediction" is a genuinely important, high-consequence distinction — the technical foundation for that later discussion is entirely the material this section has just covered.

9.5 Practical Guidance: A Checklist for Interpreting Visualisations

Bringing Sections 9.1 through 9.3 together into a practical workflow, a few guidelines are worth internalising for anyone inspecting a trained CNN's internals for the first time. When visualising kernels (Section 9.1), focus primarily on the first convolutional layer — kernels beyond the second or third layer operate on already-abstracted feature maps rather than raw pixel values, and visualised directly as image patches, they rapidly become uninterpretable to a human viewer even though they remain perfectly meaningful to the network itself. When visualising feature maps (Section 9.2), compare the same layer across several different input images rather than inspecting a single image in isolation — a feature map that appears to detect "a specific object part" on one image but nothing coherent on several others is more likely responding to an incidental pattern in that one image than encoding a genuinely reusable, meaningful feature. When using Grad-CAM (Section 9.3), always generate heatmaps for both correctly and incorrectly classified examples, not only for successes — the coarse localisation on a misclassified example is frequently more diagnostically informative than confirmation that a correct prediction looked at a sensible region, since it is precisely on failures that a spurious-correlation problem (per this section's wolves-vs-huskies callout) tends to reveal itself.

Technique	Best Applied To	Primary Question It Answers
Kernel visualisation (§9.1)	First 1–2 convolutional layers only	"What basic patterns is this layer built to detect?"
Feature map visualisation (§9.2)	Any layer, for a specific input image	"What did this layer actually detect in THIS image?"
Grad-CAM (§9.3)	Late convolutional layers, for a specific class	"Which region drove the prediction for THIS class?"

10 Research Frontiers

10.1 The Shift Toward Vision Transformers

Day 26 §10.2 previewed Vision Transformers (ViT) as a structural alternative to convolution, replacing local receptive fields and weight sharing (Section 2.4 of this day) with the self-attention mechanism. Having now developed a complete, first-principles understanding of how a CNN actually works — convolution, pooling, activations, training, regularisation, and interpretability — this day is positioned to make that forward reference considerably more concrete: a ViT processes an image as a sequence of patches, using attention to let every patch directly consider every other patch's content, rather than the strictly local, hierarchically-growing receptive field structure Section 3.3 developed for CNNs. This is a large enough architectural departure that it earns its own dedicated treatment, beginning in Day 30 and continuing in the dedicated Vision Transformers module later in this broader research programme.

10.2 Hybrid CNN-Transformer Architectures

Rather than treating CNNs and Vision Transformers as strictly competing alternatives, a substantial body of research combines both approaches within a single architecture — using convolutional layers (this day's subject) for their strong local inductive bias and computational efficiency in early stages, and attention-based layers for their ability to model long-range, global relationships in later stages. Day 30 §8 surveys this hybrid design space directly, building on the CNN foundations this day has established.

The underlying motivation for these hybrids is worth stating plainly, since it summarises a tension this day has surfaced repeatedly: a CNN's local receptive field (Section 2.4) and weight sharing impose a strong, useful inductive bias — the assumption that nearby pixels are more likely to be related than distant ones — which is a genuinely correct assumption for the vast majority of natural images and is a large part of why CNNs train effectively on comparatively modest amounts of labelled data. A Vision Transformer's attention mechanism imposes no such assumption at all, treating every patch as potentially relevant to every other patch from the very first layer, which grants it more flexibility to model genuinely long-range relationships but, correspondingly, requires far more training data to learn which relationships actually matter, since it cannot rely on any built-in locality assumption to narrow the search. Hybrid architectures attempt to capture the best of both regimes: efficient, data-friendly local processing early, flexible global reasoning late.

10.3 Efficient and Mobile CNN Design

A parallel research direction, distinct from the accuracy-focused architectural evolution Day 30 primarily covers, optimises CNNs specifically for deployment under tight computational constraints — mobile phones, embedded devices, real-time applications with strict latency budgets. MobileNet and EfficientNet (both previewed here ahead of Day 30's detailed coverage) apply architectural techniques directly built from this day's vocabulary — depthwise separable convolutions (a specific factorisation of the standard multi-channel convolution from Section 3.4, splitting it into a per-channel spatial convolution followed by a 1×1 channel-mixing convolution, dramatically reducing parameter count and compute) and principled compound scaling of depth, width, and resolution together — to achieve strong accuracy at a small fraction of a standard architecture's computational cost.

Worked Example: Depthwise Separable Convolution: A Cost Comparison

Consider transforming a 64-channel input into a 128-channel output using a 3×3 kernel, at a single spatial position.

Standard convolution: 3×3×64×128 = 73,728 multiply-accumulate operations per spatial position — every output channel depends on every input channel through a full 3×3 spatial kernel.

Depthwise separable convolution: a depthwise stage (3×3×64 = 576 operations, one small spatial kernel per input channel, no channel mixing) followed by a pointwise 1×1 stage (1×1×64×128 = 8,192 operations, per Section 3.4's 1×1 convolution). Total: 576 + 8,192 = 8,768 operations — roughly 8.4× fewer than the standard convolution's 73,728.

This roughly 8× reduction, achieved purely by factorising a single convolution into two cheaper sequential stages with no loss of output channel count, is the central mechanism behind MobileNet's efficiency, and a clear practical illustration of how the individual building blocks this day introduced (standard multi-channel convolution, §3.4; 1×1 convolution, §3.4) can be recombined into genuinely novel, more efficient architectural components.

10.4 Self-Supervised CNN Pretraining

Day 26 §10.3 previewed self-supervised pretraining as a research frontier that reduces reliance on large labelled datasets. Applied specifically to CNNs, self-supervised pretraining methods define a training objective that requires no human-provided labels at all — predicting a missing or transformed portion of an image, or, as in the contrastive and self-distillation methods behind DINO, learning representations that remain consistent across different augmented views of the same underlying image (directly building on Day 27 §6's augmentation techniques, repurposed here as the training signal itself rather than merely a regularisation aid). The resulting pretrained CNN weights can then be fine-tuned on a much smaller labelled dataset for a specific downstream task, following exactly the transfer-learning pattern Section 7.5 described, but starting from self-supervised rather than supervised pretraining — a particularly valuable option in domains (medical imaging chief among them, per Day 26 §6.3's worked example) where large labelled datasets are expensive to obtain but large unlabelled datasets are comparatively abundant.

10.5 Closing Perspective: From First Principles to the Architecture Survey

It is worth stepping back, at the close of this day's research-frontiers discussion, to state plainly what this day has actually accomplished. Every concept a modern CNN architecture is built from — the convolution operation itself (Section 2), the mechanics governing how it transforms spatial and channel dimensions (Section 3), pooling (Section 4), non-linearity (Section 5), the classic architectural template (Section 6), how such a network is actually trained (Section 7), how overfitting is controlled (Section 8), and how a trained network's decisions can be inspected and trusted (Section 9) — has now been developed from first principles, building directly and explicitly on the classical computer vision foundations Days 26 through 28 established. Day 30, starting immediately after this one, assumes fluent command of every one of these building blocks, and uses them to survey how the field progressively scaled and refined this basic template into today's much deeper, more sophisticated, and more efficient backbone architectures — the direct technical lineage from LeNet and AlexNet through VGG, ResNet, and beyond.

11 Common Pitfalls: A Practical Debugging Checklist

Because a CNN combines many interacting components — architecture (Sections 2–6), training dynamics (Section 7), and regularisation (Section 8) — a poorly performing network can have its root cause in any of several places. The following checks, in order, catch the large majority of issues encountered when a CNN trains poorly or fails to generalise.

- **Check for a shape mismatch before assuming a training-dynamics problem** — per §3.3's output-size formula, an incorrectly computed flatten dimension (Section 6.1) is one of the most common errors when building a network by hand, and produces an immediate, unambiguous error rather than a subtle performance issue — fix this first before investigating anything else.
- **Plot training AND validation loss together, not just accuracy** — per §8.1, a widening gap between training and validation loss over the course of training is the clearest direct signal of overfitting, visible well before it shows up as a validation accuracy plateau.
- **If loss is not decreasing at all, suspect the learning rate first** — per §7.4, a learning rate that is far too high causes visible divergence or oscillation; one that is far too low produces a loss curve that looks nearly flat, easily mistaken for a bug elsewhere in the pipeline.
- **Confirm dropout and batch normalization are correctly switched to evaluation mode at inference time** — per §8.2's note on this exact bug, most frameworks require an explicit mode switch (e.g. `model.eval()` in PyTorch), and forgetting it produces inconsistent, non-reproducible predictions for the identical input.
- **If a network trains well but performs poorly on genuinely new data, check for a spurious-correlation shortcut** — per §9.3's wolves-vs-huskies example, a Grad-CAM visualisation on a handful of correctly and incorrectly classified examples often reveals this kind of problem directly and quickly.
- **Verify preprocessing consistency between training and inference** — this is Day 27 §1.2 and §5.2's concern, not specific to CNNs, but worth re-checking here since it remains one of the most common causes of a model that trains well but performs unexpectedly poorly once deployed.

11.1 A Note on Reproducibility

Training a CNN involves several sources of randomness worth controlling deliberately when reproducibility matters: random weight initialisation (Section 5.4's dying-ReLU discussion depends partly on initialisation), dropout's random unit selection (Section 8.2), and data augmentation's random transformations (Section 8.5, inherited from Day 27 §6) all introduce run-to-run variation even with identical code and hyperparameters. For any comparison between two training configurations — a new regularisation technique, a different architecture, a different learning rate schedule — fixing all relevant random seeds and, ideally, averaging results across a small number of independent training runs is standard practice, directly mirroring the reproducibility discipline Day 28 §11.1 recommended for RANSAC and that the NLP-focused research days elsewhere in this programme apply to their own stochastic training runs.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Term	Definition (see section for full detail)
Kernel / Filter	A small matrix of learnable weights slid across an input to compute a feature map (§2.1, §2.3)
Feature map	The output produced by applying a kernel across an input (§2.1)
Stride	How many positions a kernel moves between successive applications (§3.1)
Padding	A border added around an input to control output size (§3.2)
Receptive field	The region of the original input a given unit is sensitive to (§3.3)
1×1 convolution	A kernel with no spatial extent, used for channel mixing (§3.4)
Max / average pooling	Downsampling operations summarising a local region by its max or mean (§4.1)
Global average pooling (GAP)	Averaging an entire feature map to one value per channel (§4.3)
ReLU	max(0, x); the dominant CNN activation function (§5.2)
Vanishing gradient	Gradients shrinking toward zero as they propagate through saturated layers (§5.2)
Backpropagation	The algorithm computing each parameter's gradient via the chain rule (§7.1)
Cross-entropy loss	The standard loss function for classification (§7.2)
Dropout	Randomly zeroing units during training to discourage co-dependence (§8.2)
Batch normalization	Normalising layer activations per mini-batch during training (§8.3)
Grad-CAM	A technique visualising which image regions drove a prediction (§9.3)

Chapter Summary: Key Takeaways

- **A convolutional kernel is mechanically identical to a classical filter** (§2.5) — the only difference is that its weights are learned via gradient descent rather than hand-designed.
- **Local receptive fields and weight sharing** (§2.4) give convolutions a roughly 2,000× parameter advantage over a comparable fully-connected layer, and are what make training on realistic dataset sizes tractable at all.
- **Stride, padding, and kernel size together determine output size** (§3.3) via a formula worth being able to apply directly; receptive field grows with network depth, not kernel size alone.
- **Non-linearity between layers is what allows depth to matter** (§5.1) — stacked linear operations collapse into a single equivalent linear operation without it.
- **ReLU dominates because it avoids the saturation that causes vanishing gradients** (§5.2) — directly setting up Day 30's residual-connection solution to the same underlying problem at greater depth.
- **Training is backpropagation + an optimiser, applied repeatedly** (§7) — and transfer learning (§7.5) is this same training loop, just initialised from pretrained weights rather than random ones.
- **Dropout, batch normalization, weight decay, and data augmentation** (§8) all combat overfitting via the same underlying logic — discourage memorisation, encourage robust, distributed representations — implemented at different points in the pipeline.
- **Grad-CAM makes CNN decisions inspectable** (§9.3) — a genuine debugging tool, not just a curiosity, and the direct technical foundation for Day 37's medical imaging explainability content.

End of Day 29 — ready for gap review and expansion into full compendium.

Theory-only day — no assigned practical project. Ready for gap review and expansion into full compendium.